C / C++

📘 Theory

1. Functions in C

- **Definition**: A block of code that performs a specific task, reusable throughout the program.
- **Types based on return type & arguments**:
  - Without return type & without arguments → void display().
  - With return type & with arguments → int add(int a, int b).
  - With return type & without arguments → double circlePI().
  - Without return type & with arguments → void multiply(int a, int b, int c).
- **Advantages**: Code reusability, modularity, easier debugging.

2. Pointers

- **Definition**: A variable that stores the memory address of another variable.
- **Operators**:
  - & → address-of operator.
  - * → dereference operator.
- **Pass by value vs pass by reference**:
  - Pass by value → function gets a copy of variable.
  - Pass by reference → function gets the actual memory address, so changes persist.

3. Swap Example

- **Pass by value**: Values don't change outside the function.
- **Pass by reference**: Values swap correctly because memory addresses are modified.

Coding Round Questions

- Functions → Explain different types of functions in C with examples.
- Return type → What is the difference between void and int functions?
- Pointers → Explain pointers and their use in functions.
- Pass by reference → Show how swapping works using pointers.
- Dereferencing → Explain dereferencing with an example.
- Function reusability → Why are functions important for modular programming?

⚡ **Machine Coding Round Challenge**

Problem: Calculator with Functions & Pointers

Requirements:

1. Create functions for addition, subtraction, multiplication, and division.
2. Use pointers to pass values into functions.
3. Display results for each operation.
4. Demonstrate both **pass by value** and **pass by reference**.

```c
#include <stdio.h>


void display(int *b){

    *b = 100;
    printf("inside function : %d \n", b);
}

void swap(int *a, int *b){

    int temp = *a;
    *a = *b; // 30
    *b = temp; // 10
}
// void swap(int a, int b){
//     int temp = a;
//     a = b;
//     b = temp;
```

```c
// }

int main(){
    // pointers => stores another variables memory address
    int a = 10;
    int b = 30;
    printf("Before swap a and b value %d , %d \n", a, b);

    swap(&a, &b);
    // swap(a, b);

    printf("After swap a and b value %d , %d \n", a, b);

    // printf("Before function : %d \n",a);
    // display(&a);

    // printf("After function : %d \n",a);
    int *ptr = &a;

    printf("%p \n" , ptr);
    printf("%d \n", *ptr);  // deferencing
    // pass by value and pass by reference
    return 0;
}
```

```c
#include <stdio.h>


// without return type and without argument
void display(){ // call definition
    printf("Welcome to our website \n");
}
// with return type and with argument
int add(int a , int b){ // function definition (parameter)
    int sum = a+b;
    return sum;
}
// with return type and without argument
double circlePI(){ // function definition
    return 3.14;
}

// without return type and with argument

void multiply(int a, int b, int c){
```

```c
    int result = a *b *c;
    printf("Multiply value : %d \n", result);
}

int main(){

    printf("Hello Functions \n");
    // calling a function
    display();

    int result = add(10,10); // (argument) // 20
    printf("%d \n", result);

    double PI = circlePI();
    printf("%lf \n", PI);

    multiply(10,20,30);

    // functions  => reusable block of code
    return 0;
}
```